

The Modern Software Developer

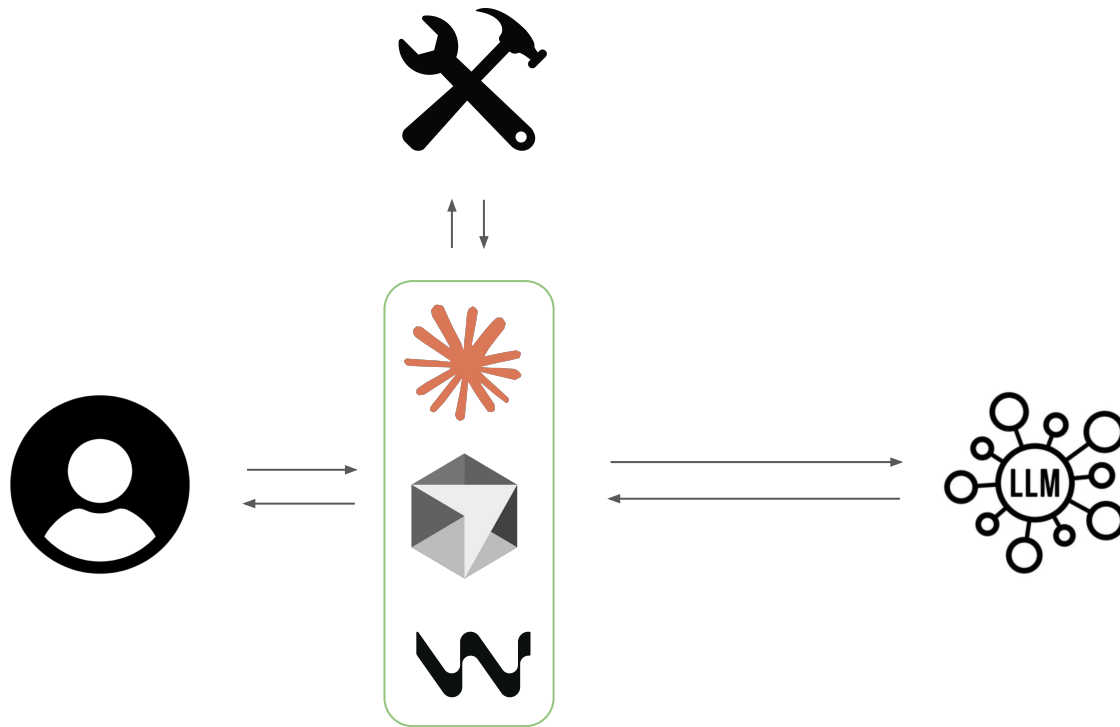
CS146S
Stanford University, Fall 2026
Mihail Eric

Logistics

- Final project specifications out
 - Allowed 1 additional team member (in addition to a swarm of coding agents)
- Credits for different coding agent providers
 - Claude
 - Factory
 - [X.AI](#)
 - Devin
- More details on OSS contributions next week
- Final deadline is 9/27 for course applications

The Anatomy of a Modern Coding Agent

Recap

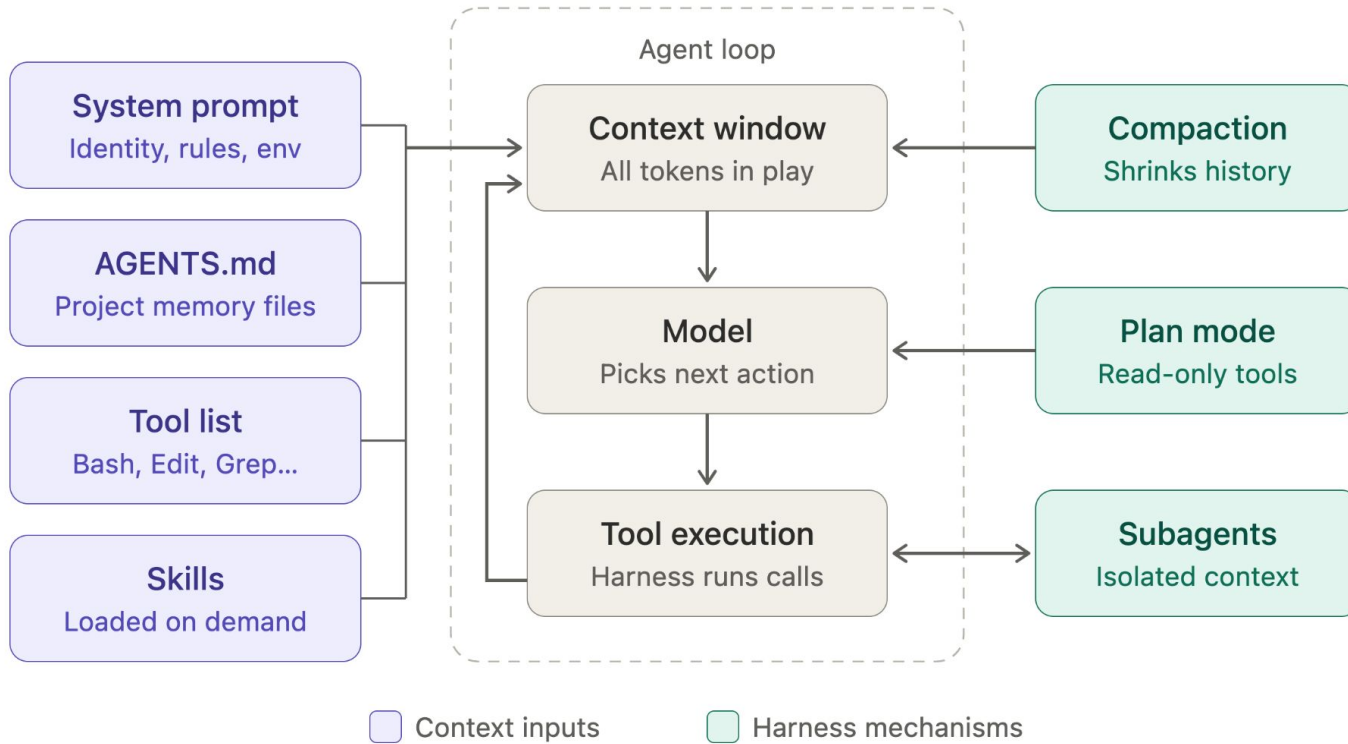


Enter a state-of-the-art coding agent

	Opus 5.5	Fable 5.1	Opus 5	GPT-6 Astra	GPT-5.6 Sol
Agentic coding Terminal-Bench 4.0 ¹	66.4%	55.8%	52.3%	57.9%	37.3%
Agentic coding FrontierCode v1.1 (Main)	54.4%	50.3%	48.0%	53.3%	47.5%
Agentic coding CursorBench 4.0	57.8%	51.8%	46.6%	—	41.7%
Knowledge work GDPval-AA v2.1	1846	1735	1708	1542	1588
Business workflows AutomationBench ²	40.0%	31.4%	26.9%	41.4%	28.8%
Multidisciplinary reasoning Humanity's Last Exam	67.7% with tools	65.6% with tools	63.6% with tools	57.2% with tools	—

Modern coding harnesses are more complex

- Harness design is one of the most important problems in coding agents
 - Components
 - System prompt
 - Custom configuration files ([AGENTS.md](#), etc)
 - Tool list
 - Plan mode
 - Subagents
 - Skills
 - Hooks
 - Compaction



Tool List

- Lean but powerful capabilities across search, shell commands, agent tasks
 - ~40 built-in tools in the case of Claude
 - MCP tools are also available and loaded into context
 - Description, names, full signature is a part of the prompt

System Prompt

- System level blueprint
- Defines
 - Harness / behaviors
 - Guardrails
 - Environment (ie what is the system, where are we)

Agent Configuration Files

- User-level agent customizations like [CLAUDE.md/AGENTS.md](#)
- Take precedence over other commands

Plan Mode

- Explicit tool prompting agent to make a spec for a change
- Can engage in further questioning with user to clarify requirements

Skills

- Describes a written procedure to be performed by agent
 - A prompt with tool calls and/or local script invocations
- Can be invoked proactively or explicitly

Subagents

- Invoked via an explicit tool call
- Fresh context within session with custom system prompt
- Allows for parallelization (though token costs go up)
- Final result passed back to parent

Hooks

- Prompts or snippets of code that are inserted at different points in the harness lifecycle
- Especially interesting if some pre/post-processing needs to get done
 - We'll discuss best practices later

Compaction

- Create consolidated summary of key points in agent run when context is almost exceeded
- Replace existing history with summary
- Triggered manually or automatically
- Can be lossy so you may want to adapt your own

Principles of Effective Harnesses

- Simplicity
 - Fewer, more powerful tools are preferred (sql, bash, etc)
- Mind the context window – avoid the dumb zone so be careful of tool bloat
 - More on optimal context engineering later
- Prompt contents and lengths will evolve – make it easy to adapt them

Questions?